parameter), this can backfire: all it takes is for two threads to try to exchange data between the same two instances with the parameters swapped, and you have deadlock!

Thankfully, the C++ Standard Library has a cure for this in the form of `std::lock`—a function that can lock two or more mutexes at once without risk of deadlock. The example in the next listing shows how to use this for a simple swap operation.

Listing 3.6. Using `std::lock()` and `std::lock_guard` in a swap operation

```
class some_big_object;
void swap(some_big_object& lhs,some_big_object& rhs);

class X
{
private:
    some_big_object some_detail;
    std::mutex m;
public:
    X(some_big_object const& sd):some_detail(sd){}

    friend void swap(X& lhs, X& rhs)
    {
        if(&lhs==&rhs)
            return;
        std::lock(lhs.m,rhs.m);                                          // 1
        std::lock_guard<std::mutex> lock_a(lhs.m,std::adopt_lock);       // 2
        std::lock_guard<std::mutex> lock_b(rhs.m,std::adopt_lock);       // 3
        swap(lhs.some_detail,rhs.some_detail);
    }
};
```

First, the arguments are checked to ensure they are different instances, because attempting to acquire a lock on a `std::mutex` when you already hold it is undefined behavior. (A mutex that does permit multiple locks by the same thread is provided in the form of `std::recursive_mutex`. See section 3.3.3 for details.) Then, the call to `std::lock()` ❶ locks the two mutexes, and two `std::lock_guard` instances are constructed ❷, ❸, one for each mutex. The `std::adopt_lock` parameter is supplied in addition to the mutex to indicate to the `std::lock_guard` objects that the mutexes are already locked, and they should just adopt the ownership of